R&S-type

26.09.03 정명훈

목차

1.Project Overview

- 개요
- 목표

2.R-type

- Case Table
- Case Simulation

3.S-type

- Case Table
- Case simulation

4.Review

- 고찰 및 결론

1.Project Overview

개요

- R-type 명령어의 주요 연산 Edge case 및 S-type 명령어를 System Verilog로 설계 및 동작 검증
- Instruction ROM, Register File, Control Unit, ALU, Data Memory 간의 명령어 인코딩/디코딩 및 제어 신호 흐름 follow

목표

- R-type 각 명령어 구현 및 명령어의 기본 동작과 Edge case 검증
- S-type 명령어 실행 시 ALU가 연산한 데이터 주소와 데이터 메모리의 매핑 검증

2.R-type

Normal Case Table

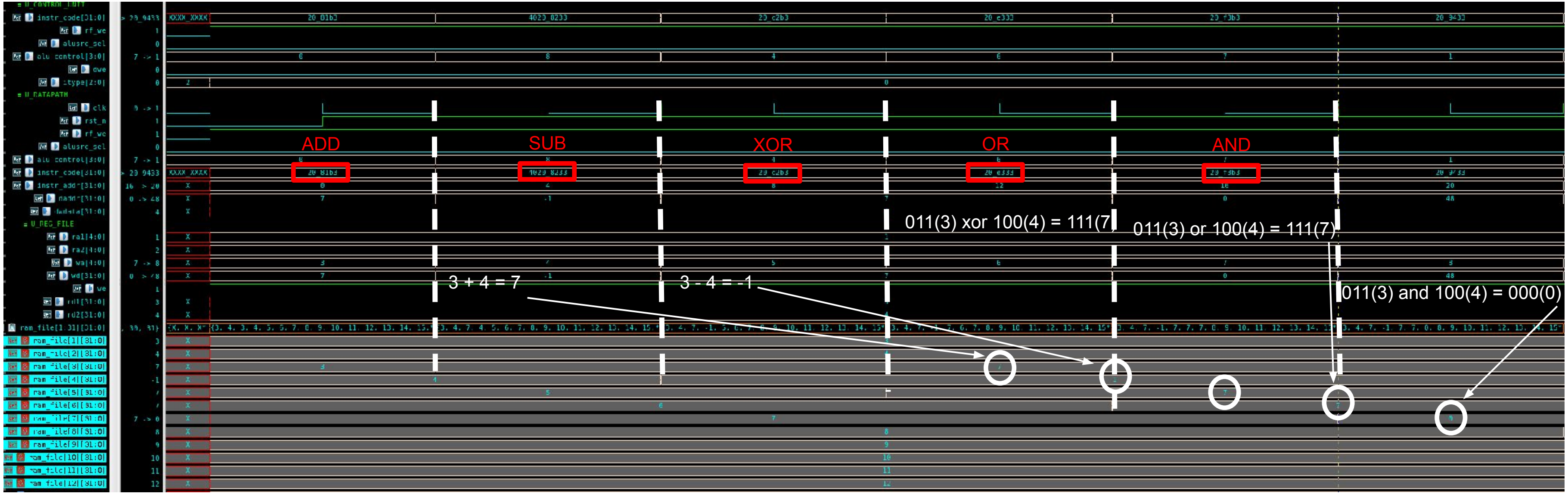
- 32bit의 값을 4bit씩 끊어 hex값으로 instruction rom에 저장

Inst	fucn7	rs2	rs1	func3	rd	opcode	instr_rom
ADD	0 0000_000 0	2 0_0010	0 0_0001	8 000	1 0_0011	3 b 011_0011	h'0020_81b3
SUB	0100_000	0_0010	0_0001	000	0_0100	011_0011	h'4020_8233
XOR	0000_000	0_0010	0_0001	100	0_0101	011_0011	h'0020_c2b3
OR	0000_000	0_0010	0_0001	110	0_0110	011_0011	h'0020_e333
AND	0000_000	0_0010	0_0001	111	0_0111	011_0011	h'0020_f3b3
SLL	0000_000	0_0010	0_0001	001	0_1000	011_0011	h'0020_9433
SRL	0000_000	0_0010	0_0001	101	0_1001	011_0011	h'0020_d4b3
SRA	0100_000	0_0010	0_0001	101	0_1010	011_0011	h'4020_d53

2.R-type

Normal Case Simulation(ADD, SUB, XOR, OR, AND)

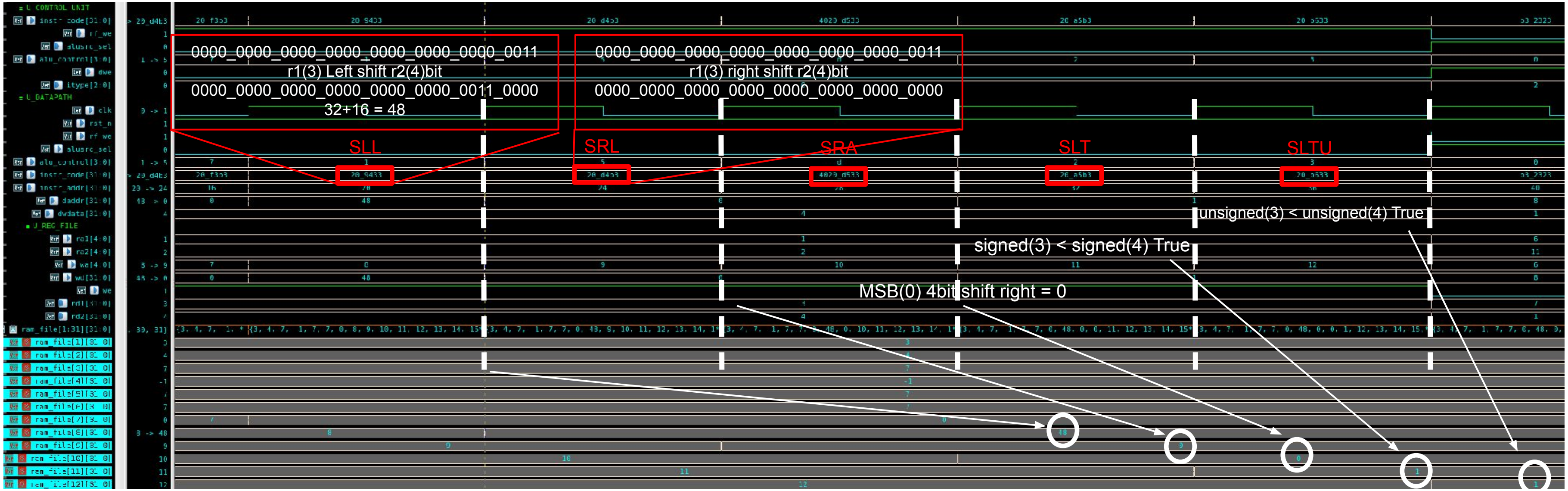
- Register의 [1] = 3, [2] = 4로 초기화 시켜 result(wd) = rs1(reg[1]) alu rs2(reg[2])을 진행
- 결과(wd)를 address 3번째부터 1씩 증가하여 총 7번째 address까지 저장
- Instruction 수행 후 다음 clk의 rising edge에 register file에 값 저장


U_CONTROL_UNIT
instr_code[31:0]
rf_we
alusrc_sel
alu_control[3:0]
dwe
itype[2:0]
U_DATAPATH
clk
rst_n
rf_we
alusrc_sel
alu_control[3:0]
instr_code[31:0]
instr_addr[31:0]
daddr[31:0]
ddata[31:0]
U_REG_FILE
ra1[4:0]
ra2[4:0]
wa[4:0]
wd[31:0]
we
rd1[31:0]
rd2[31:0]
rom_file[1:31][31:0]
ADD
SUB
XOR
OR
AND
3 + 4 = 7
3 - 4 = -1
011(3) xor 100(4) = 111(7)
011(3) or 100(4) = 111(7)
011(3) and 100(4) = 000(0)

2.R-type

Normal Case Simulation(SLL, SRL, SRA, SLT, SLTU)

- 결과(wd)를 address 8번째부터 1씩 증가하여 총 12번째 address까지 저장
- Register의 [1] = 3, [2] = 4로 초기화 시켜 result(wd) = rs1(reg[1]) alu rs2(reg[2])을 진행
- Instruction 수행 후 다음 clk의 rising edge에 register file에 값 저장



0000_0000_0000_0000_0000_0000_0000_0011
r1(3) Left shift r2(4)bit
0000_0000_0000_0000_0000_0000_0011_0000
32+16 = 48

0000_0000_0000_0000_0000_0000_0000_0011
r1(3) right shift r2(4)bit
0000_0000_0000_0000_0000_0000_0000_0000

unsigned(3) < unsigned(4) True

signed(3) < signed(4) True

MSB(0) 4bit shift right = 0

2.R-type

Edge Case Table

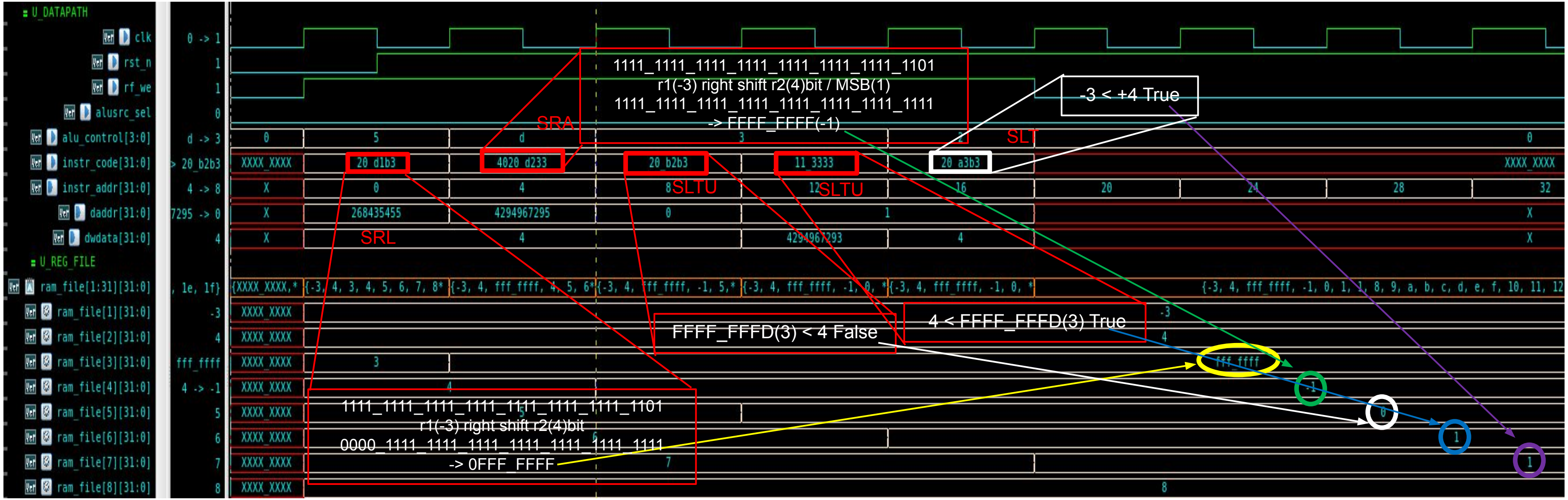
- 32bit의 값을 4bit씩 끊어 hex값으로 instruction rom에 저장
- SRL: 음수값을 4bit shift right 할 경우 최상위 bit에 0이 채워지는지 확인
- SLTU: Unsigned 음수와 양수를 바꿔 비교하여 확인
- SLT: Signed 음수와 양수를 비교하여 확인

Inst	fucn7	rs2	rs1	func3	rd	opcode	instr_rom
SRL	0 0000_000	0 2 0_0010	0 0_0001	d 101	1 0_0011	b 3 011_0011	h'0020_d1b3
SRA	0100_000	0_0010	0_0001	101	0_0100	011_0011	h'4020_d233
SLTU	0000_000	0_0010	0_0001	011	0_0101	011_0011	h'0020_b2b3
SLTU	0000_000	0_0001	0_0010	011	0_0110	011_0011	h'0011_3333

2.R-type

Edge Case Simulation(SRL, SRA, SLTU, SLT)

- Register의 [1] = -3, [2] = 4로 초기화 시켜 result(wd) = rs1(reg[1]) alu rs2(reg[2])을 진행
- 결과(wd)를 address 3번째부터 1씩 증가하여 총 7번째 address까지 저장
- Instruction 수행 후 다음 clk의 rising edge에 register file에 값 저장


U_DATAPATH
clk
rst_n
rf_we
alusrc_sel
alu_control[3:0]
instr_code[31:0]
instr_addr[31:0]
daddr[31:0]
dwdata[31:0]
U_REG_FILE
ram_file[1:31][31:0]
ram_file[1][31:0]
ram_file[2][31:0]
ram_file[3][31:0]
ram_file[4][31:0]
ram_file[5][31:0]
ram_file[6][31:0]
ram_file[7][31:0]
ram_file[8][31:0]
20 d1b3
4020 d233
20 b2b3
11 3333
20 a3b3
XXXX XXXX
268435455
4294967295
4294967293
{-3, 4, fff_ffff, -1, 0, 1, 1, 8, 9, a, b, c, d, e, f, 10, 11, 12
fff_ffff

SRA

1111_1111_1111_1111_1111_1111_1111_1101
r1(-3) right shift r2(4)bit / MSB(1)
1111_1111_1111_1111_1111_1111_1111_1111
-> FFFF_FFFF(-1)

SLT

-3 < +4 True

SLTU

SLTU

SRL

FFFF_FFFD(3) < 4 False

4 < FFFF_FFFD(3) True

1111_1111_1111_1111_1111_1111_1111_1101
r1(-3) right shift r2(4)bit
0000_1111_1111_1111_1111_1111_1111_1111
-> 0FFF_FFFF

3.S-type

Case Table

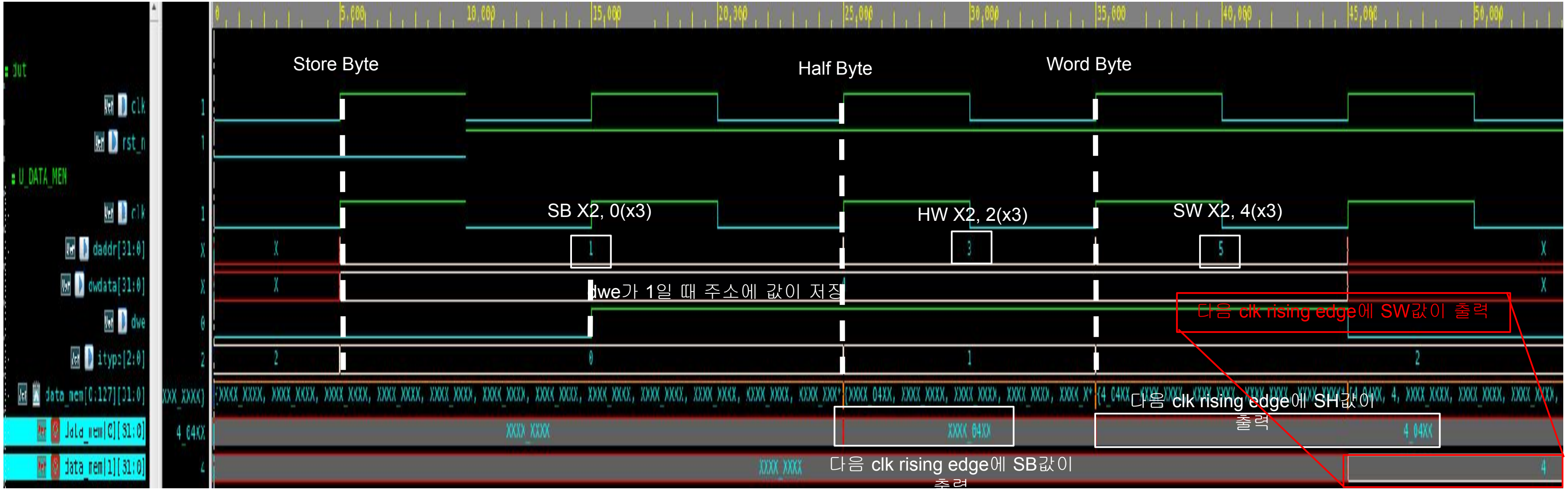
- 32bit의 값을 4bit씩 끊어 hex값으로 instruction rom에 저장
- -3+4 연산을 진행 후 결과값 1을 x3에 저장하여 주소값 계산 시 rd0에 x3값 add
- SB: x2, 0(x3=1) -> 하위 2bit slice -> data_mem[0] | daddr[1:0] = 2'b01 -> byte[15:8]에 주소값 저장
- SH: x2, 2(x3=1) -> 하위 2bit slice -> data_mem[0] | daddr[1] = 1b'1 -> byte[31:16]에 주소값 저장
- SW: x2, 4(x3=1) -> 하위 2bit slice -> data_mem[1] | 32bit 전체 쓰기 -> 32'h0000_0004 저장

Inst	imm	rs2	rs1	func3	imm	opcode	instr_rom
SB	0 0000_000	0 2 0_0010	1 0_0011	8 000	0 0_0000	2 3 010_0011	h'0021_8023
SH	0000_000	0_0010	0_0011	001	0_0010	010_0011	h'0021_9123
SW	0000_000	0_0010	0_0001	100	0_0100	010_0011	h'0021_a223

3.S-type

Case Simulation(SB, SH, SW)

- Register의 [1] = -3, [2] = 4로 초기화 시켜 wd(1) = rs1(-3) add rs2(4)을 진행
- 각 SB, SH, SW의 결과(dwdata)를 address(rs1(x3)+rs2(imm)) 저장
- Instruction 수행 후 다음 clk의 rising edge에 register file에 값 저장


0
5,000
10,000
15,000
20,000
25,000
30,000
35,000
40,000
45,000
50,000
dut
clk
rst_n
U_DATA_MEM
clk
daddr[31:0]
dwdata[31:0]
dwe
itype[2:0]
data_mem[0:127][31:0]
data_mem[0][31:0]
data_mem[1][31:0]
1
1
1
X
X
0
2
XXX XXXX}
4 04XX
4
X
X
1
3
5
2
0
1
2
XXXX XXXX
XXXX 04XX
4 04XX
XXXX XXXX
4

Store Byte

Half Byte

Word Byte

SB X2, 0(x3)

HW X2, 2(x3)

SW X2, 4(x3)

dwe가 1일 때 주소에 값이 저장

다음 clk rising edge에 SW값이 출력

다음 clk rising edge에 SH값이 출력

다음 clk rising edge에 SB값이 출력

4.Review

고찰

- ALU 내부에서 음수 데이터 처리 시 SRL, SRA 동작 차이 및 SLT, SLTU edge 조건 판단 확인
- ALU에서 daddr=rs1(x3)+imm 연산을 통해 각 SB,SH,SW의 바이트 주소를 추출

결론

- Control Unit의 제어 신호와 ALU 주소 계산, Data Memory bit slicing이 동작함을 확인
- Waveform을 통해 Register file 및 Data Memory가 simulation scenario에 맞게 값이 저장 및 유지됨을 확인